



JDBC: Connectivity with Database

Section 1: Learn

Setting Up JDBC Environment

To connect Java with a database (like MySQL or Oracle), you need:

1. **JDK installed**
 2. **Database installed and running** (e.g., MySQL)
 3. **JDBC Driver** (e.g., mysql-connector-java.jar)
-

Steps to Set Up JDBC:

1. **Download JDBC driver** for your DBMS.
 - For MySQL: <https://dev.mysql.com/downloads/connector/j/>
 2. **Add JDBC driver JAR to your project:**
 - In Eclipse: Right-click project > Build Path > Add External JARs
 3. **Use correct connection details:**
 - **URL:** `jdbc:mysql://localhost:3306/dbname`
 - **Username:** e.g., `root`
 - **Password:** e.g., `admin123`
-

Establishing a Connection Using DriverManager

`DriverManager` is a built-in class in `java.sql` package that helps establish a connection.

Steps to Connect:

```
import java.sql.*;
```



```
public class DBConnect {  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/student_db";  
        String user = "root";  
        String password = "admin123";  
  
        try {  
            Connection conn = DriverManager.getConnection(url, user,  
password);  
            System.out.println("Connection Successful!");  
            conn.close();  
        } catch (SQLException e) {  
            System.out.println("Connection Failed!");  
            e.printStackTrace();  
        }  
    }  
}
```

Handling Exceptions in JDBC

When using JDBC, most issues arise from **SQLException**.

Common Causes:

- Incorrect URL
- Wrong credentials
- Driver not found
- Database not running



Best Practices for Handling:

- Use **try-catch-finally**
 - Use `e.getMessage()` and `e.printStackTrace()` for details
 - Close all resources in the **finally** block
-

Sample Code with Exception Handling

```
Connection conn = null;

try {
    conn = DriverManager.getConnection(url, user, password);
    System.out.println("Connected to DB");
} catch (SQLException e) {
    System.out.println("SQL Exception occurred: " + e.getMessage());
} finally {
    try {
        if (conn != null) conn.close();
    } catch (SQLException e) {
        System.out.println("Error closing connection: " + e.getMessage());
    }
}
```



Section 2: Practice

Set Up Your Own DB Connection

1. Install MySQL or use online DB tools
 2. Create a DB and user table
 3. Write Java code to connect using JDBC
-

Section 3: Know More (FAQs)

What is DriverManager?

It's a utility class in JDBC used to manage database drivers and connections.

Do I need to load the driver manually?

No, from **JDBC 4.0 onwards**, drivers are loaded automatically when the JAR is added.

What port does MySQL run on?

Default is **3306**.

Can I connect to remote databases?

Yes. Just change the URL to point to the **remote server IP or domain**.

What if the database is down?

A **SQLException** will be thrown during the connection attempt.